Essentials_of_Linux

October 9, 2025

1 Omics — Linux Essentials (Notebook)

Department of Biotechnology
University of Tehran
Fall 2025

Content Creator: Mahdi Anvari

Welcome!
This notebook is a **comprehensive resource** for the Linux Essentials sessions in the Omics course. It collects **core Linux commands, explanations, examples, and small exercises** for interactive learning.

What's inside - Navigation commands (`pwd`, `cd`, `ls`, etc.)
- File and directory management (`touch`, `mkdir`, `cp`, `mv`, `rm`, `rmdir`)
- Viewing and processing file contents (`cat`, `head`, `tail`, `less`, `wc`, `grep`)
- System and user management (`whoami`, `sudo`, `history`, `alias`)
- Downloading and compressing files (`wget`, `gzip`, `tar`)

Safety Tips - Avoid running destructive commands (e.g., `rm -rf`) on important folders
- Use temporary directories or test files to experiment safely

> Tip: Try modifying the commands in the code cells to see how options change the output. Experimentation is a safe way to learn!

1.1 1. Getting Help — `man`

The `man` (manual) command displays **documentation for other commands** right in the terminal. It's one of the most useful tools for exploring Linux commands and their options.

Basic syntax: "'bash man [command]

[8]:
```
man ls
```

```
LS(1)                          User Commands                          LS(1)

NAME
       ls - list directory contents

SYNOPSIS
```

```
       ls [OPTION]… [FILE]…

DESCRIPTION
       List  information  about  the FILEs (the current directory by default).
       Sort entries alphabetically if none of -cftuvSUX nor --sort  is  speci-
       fied.

       Mandatory  arguments  to  long  options are mandatory for short options
       too.

       -a, --all
              do not ignore entries starting with .

       -A, --almost-all
              do not list implied . and ..

       --author
              with -l, print the author of each file

       -b, --escape
              print C-style escapes for nongraphic characters

       --block-size=SIZE
              with  -l,  scale  sizes  by  SIZE  when  printing  them;   e.g.,
              '--block-size=M'; see SIZE format below

       -B, --ignore-backups
              do not list implied entries ending with ~

       -c     with -lt: sort by, and show, ctime (time of last modification of
              file status information); with -l: show ctime and sort by  name;
              otherwise: sort by ctime, newest first

       -C     list entries by columns

       --color[=WHEN]
              colorize  the output; WHEN can be 'always' (default if omitted),
              'auto', or 'never'; more info below

       -d, --directory
              list directories themselves, not their contents

       -D, --dired
              generate output designed for Emacs' dired mode

       -f     do not sort, enable -aU, disable -ls --color

       -F, --classify
```

```
       append indicator (one of */=>@|) to entries

--file-type
       likewise, except do not append '*'

--format=WORD
       across -x, commas -m, horizontal -x, long -l, single-column  -1,
       verbose -l, vertical -C

--full-time
       like -l --time-style=full-iso

-g     like -l, but do not list owner

--group-directories-first
       group directories before files;

       can  be  augmented  with  a  --sort  option,  but  any  use  of
       --sort=none (-U) disables grouping

-G, --no-group
       in a long listing, don't print group names

-h, --human-readable
       with -l and -s, print sizes like 1K 234M 2G etc.

--si   likewise, but use powers of 1000 not 1024

-H, --dereference-command-line
       follow symbolic links listed on the command line

--dereference-command-line-symlink-to-dir
       follow each command line symbolic link

       that points to a directory

--hide=PATTERN
       do not list implied entries matching shell  PATTERN  (overridden
       by -a or -A)

--hyperlink[=WHEN]
       hyperlink file names; WHEN can be 'always' (default if omitted),
       'auto', or 'never'

--indicator-style=WORD
       append indicator with style WORD to entry names: none (default),
       slash (-p), file-type (--file-type), classify (-F)
```

```
-i, --inode
       print the index number of each file

-I, --ignore=PATTERN
       do not list implied entries matching shell PATTERN

-k, --kibibytes
       default  to  1024-byte  blocks for disk usage; used only with -s
       and per directory totals

-l     use a long listing format

-L, --dereference
       when showing file information for a symbolic link, show informa-
       tion  for  the file the link references rather than for the link
       itself

-m     fill width with a comma separated list of entries

-n, --numeric-uid-gid
       like -l, but list numeric user and group IDs

-N, --literal
       print entry names without quoting

-o     like -l, but do not list group information

-p, --indicator-style=slash
       append / indicator to directories

-q, --hide-control-chars
       print ? instead of nongraphic characters

--show-control-chars
       show nongraphic characters as-is (the default, unless program is
       'ls' and output is a terminal)

-Q, --quote-name
       enclose entry names in double quotes

--quoting-style=WORD
       use  quoting style WORD for entry names: literal, locale, shell,
       shell-always,  shell-escape,  shell-escape-always,   c,   escape
       (overrides QUOTING_STYLE environment variable)

-r, --reverse
       reverse order while sorting
```

```
-R, --recursive
       list subdirectories recursively

-s, --size
       print the allocated size of each file, in blocks

-S     sort by file size, largest first

--sort=WORD
       sort  by  WORD instead of name: none (-U), size (-S), time (-t),
       version (-v), extension (-X)

--time=WORD
       change the default of  using  modification  times;  access  time
       (-u): atime, access, use; change time (-c): ctime, status; birth
       time: birth, creation;

       with -l, WORD determines which time to show;  with  --sort=time,
       sort by WORD (newest first)

--time-style=TIME_STYLE
       time/date format with -l; see TIME_STYLE below

-t     sort by time, newest first; see --time

-T, --tabsize=COLS
       assume tab stops at each COLS instead of 8

-u     with  -lt:  sort by, and show, access time; with -l: show access
       time and sort by name; otherwise: sort by  access  time,  newest
       first

-U     do not sort; list entries in directory order

-v     natural sort of (version) numbers within text

-w, --width=COLS
       set output width to COLS.  0 means no limit

-x     list entries by lines instead of by columns

-X     sort alphabetically by entry extension

-Z, --context
       print any security context of each file

-1     list one file per line.  Avoid '\n' with -q or -b
```

--help display this help and exit

--version
output version information and exit

The SIZE argument is an integer and optional unit (example: 10K is 10*1024). Units are K,M,G,T,P,E,Z,Y (powers of 1024) or KB,MB,… (powers of 1000). Binary prefixes can be used, too: KiB=K, MiB=M, and so on.

The TIME_STYLE argument can be full-iso, long-iso, iso, locale, or +FORMAT. FORMAT is interpreted like in date(1). If FORMAT is FORMAT1<newline>FORMAT2, then FORMAT1 applies to non-recent files and FORMAT2 to recent files. TIME_STYLE prefixed with 'posix-' takes effect only outside the POSIX locale. Also the TIME_STYLE environment variable sets the default style to use.

Using color to distinguish file types is disabled both by default and with --color=never. With --color=auto, ls emits color codes only when standard output is connected to a terminal. The LS_COLORS environment variable can change the settings. Use the dircolors command to set it.

Exit status:
0 if OK,

1 if minor problems (e.g., cannot access subdirectory),

2 if serious trouble (e.g., cannot access command-line argument).

AUTHOR
Written by Richard M. Stallman and David MacKenzie.

REPORTING BUGS
GNU coreutils online help: <https://www.gnu.org/software/coreutils/>
Report any translation bugs to <https://translationproject.org/team/>

COPYRIGHT

SEE ALSO
Full documentation <https://www.gnu.org/software/coreutils/ls>
or available locally via: info '(coreutils) ls invocation'

1.2 2. Listing Files and Directories — `ls`

The `ls` command is used to **list the contents** of a directory.

Basic syntax: “‘bash ls [options] [directory]

[2]:
```
ls
```

```
EssentialsOfLinux.ipynb
```

[3]:
```
ls ~
```

```
'MEGA X'           Untitled2.ipynb   env1.yml
miniconda3   perl5
 Untitled.ipynb   anaconda3         igv
mydir        temp-libraries
```

[4]:
```
ls -l ~
```

```
total 40
drwxr-xr-x  3 mahdi mahdi 4096 Mar  2  2025 'MEGA X'
-rw-r--r--  1 mahdi mahdi  992 Sep 27 09:53  Untitled.ipynb
-rw-r--r--  1 mahdi mahdi 3042 Sep 15 17:15  Untitled2.ipynb
drwxr-xr-x 31 mahdi mahdi 4096 Mar 12  2025  anaconda3
-rw-r--r--  1 mahdi mahdi 1312 Jun  4  2024  env1.yml
drwxr-xr-x  5 mahdi mahdi 4096 Jul  3 21:36  igv
drwxr-xr-x 19 mahdi mahdi 4096 Dec 25  2024  miniconda3
drwxr-xr-x  3 mahdi mahdi 4096 Aug 18 20:24  mydir
drwxr-xr-x  5 mahdi mahdi 4096 Jul  3  2024  perl5
drwxr-xr-x  4 mahdi mahdi 4096 Dec 22  2024  temp-libraries
```

[6]:
```
ls -ahl ~
```

```
total 684K
drwxr-x--- 35 mahdi mahdi 4.0K Oct  9 11:30  .
drwxr-xr-x  3 root  root  4.0K Dec 15  2023  ..
drwxr-x---  2 mahdi mahdi 4.0K Apr  7  2024  .android
-rw-------  1 mahdi mahdi  41K Oct  6 10:00  .bash_history
-rw-r--r--  1 mahdi mahdi  220 Dec 15  2023  .bash_logout
-rw-r--r--  1 mahdi mahdi 5.3K May 21 10:51  .bashrc
drwx------ 17 mahdi mahdi 4.0K Aug 18 19:19  .cache
drwxr-xr-x  7 mahdi mahdi 4.0K Feb 25  2024  .codon
drwxr-xr-x  2 mahdi mahdi 4.0K Jan  7  2024  .conda
-rw-r--r--  1 mahdi mahdi   52 Jun  4  2024  .condarc
drwxr-xr-x 14 mahdi mahdi 4.0K Jun 28 14:24  .config
drwxr-xr-x  6 mahdi mahdi 4.0K Jul  3  2024  .cpan
drwxr-xr-x  4 mahdi mahdi 4.0K Jul  3  2024  .cpanm
drwx------  3 mahdi mahdi 4.0K Oct  6 09:45  .dbus
drwxr-xr-x  3 mahdi mahdi 4.0K May  4 17:01  .dotnet
```

```
-rw-r--r--  1 mahdi mahdi  186 Jul 26 13:27  .gitconfig
drwxr-xr-x  2 mahdi mahdi 4.0K Mar 28  2024  .igv
drwxr-xr-x  2 mahdi mahdi 4.0K Sep 15 17:14  .ipynb_checkpoints
drwxr-xr-x  3 mahdi mahdi 4.0K Apr 20  2024  .ipython
drwxr-xr-x  4 mahdi mahdi 4.0K Mar 28  2024  .java
drwxr-xr-x  3 mahdi mahdi 4.0K Aug 16  2024  .jupyter
drwxr-xr-x  2 mahdi mahdi 4.0K Dec 23  2024  .landscape
-rw-------  1 mahdi mahdi   20 Oct  6 09:24  .lesshst
drwxr-xr-x  3 mahdi mahdi 4.0K Dec 21  2023  .local
-rw-r--r--  1 mahdi mahdi    0 Aug 18 12:25  .motd_shown
drwx------  2 mahdi mahdi 4.0K Feb 19  2024  .ncbi
drwxr-xr-x  4 mahdi mahdi 4.0K Mar 29  2024  .npm
drwx------  3 mahdi mahdi 4.0K Jan  7  2024  .nv
drwxr-xr-x 13 mahdi mahdi 4.0K Aug 18 18:56  .oh-my-zsh
drwx------  3 mahdi mahdi 4.0K Mar  2  2025  .pki
-rw-r--r--  1 mahdi mahdi  897 Feb 25  2024  .profile
-rw-------  1 mahdi mahdi  194 Oct 27  2024  .python_history
-rw-r--r--  1 mahdi mahdi   10 Aug 18 18:56  .shell.pre-oh-my-zsh
drwxr-xr-x  7 mahdi mahdi 4.0K Mar 20  2025  .sos
drwx------  2 mahdi mahdi 4.0K Feb 12  2025  .ssh
-rw-r--r--  1 mahdi mahdi    0 Dec 15  2023  .sudo_as_admin_successful
-rw-------  1 mahdi mahdi  12K Dec 18  2023  .swp
drwxr-xr-x  2 mahdi mahdi 4.0K Dec 22  2024  .vcpkg
-rw-------  1 mahdi mahdi  22K Oct  4 09:40  .viminfo
-rw-r--r--  1 mahdi mahdi   70 Apr  5  2024  .vimrc
drwxr-xr-x  3 mahdi mahdi 4.0K May  4 17:01  .vscode-remote-
containers
drwxr-xr-x  5 mahdi mahdi 4.0K May  4 17:01  .vscode-server
-rw-r--r--  1 mahdi mahdi  628 Jul 20 09:54  .wget-hsts
-rw-r--r--  1 mahdi mahdi  55K Oct  9 11:17  .zcompdump
-rw-r--r--  1 mahdi mahdi  50K Aug 18 19:57  .zcompdump-DESKTOP-PIJCDCB-5.8.1
-r--r--r--  1 mahdi mahdi 115K Aug 18 19:57  .zcompdump-DESKTOP-
PIJCDCB-5.8.1.zwc
-rw-r--r--  1 mahdi mahdi  50K Aug 18 19:58  .zcompdump-x86_64-conda-linux-
gnu-5.8.1
-r--r--r--  1 mahdi mahdi 115K Aug 18 19:58  .zcompdump-x86_64-conda-linux-
gnu-5.8.1.zwc
-rw-------  1 mahdi mahdi 6.1K Oct  9 11:30  .zsh_history
-rw-r--r--  1 mahdi mahdi 5.2K Aug 18 19:58  .zshrc
drwxr-xr-x  3 mahdi mahdi 4.0K Mar  2  2025 'MEGA X'
-rw-r--r--  1 mahdi mahdi  992 Sep 27 09:53  Untitled.ipynb
-rw-r--r--  1 mahdi mahdi 3.0K Sep 15 17:15  Untitled2.ipynb
drwxr-xr-x 31 mahdi mahdi 4.0K Mar 12  2025  anaconda3
-rw-r--r--  1 mahdi mahdi 1.3K Jun  4  2024  env1.yml
drwxr-xr-x  5 mahdi mahdi 4.0K Jul  3 21:36  igv
drwxr-xr-x 19 mahdi mahdi 4.0K Dec 25  2024  miniconda3
drwxr-xr-x  3 mahdi mahdi 4.0K Aug 18 20:24  mydir
```

```
drwxr-xr-x  5 mahdi mahdi 4.0K Jul  3  2024  perl5
drwxr-xr-x  4 mahdi mahdi 4.0K Dec 22  2024  temp-libraries
```

1.3 3. Navigating Directories — cd

The cd command (**change directory**) is used to move between directories in the Linux filesystem.

Basic syntax: “‘bash cd [directory]

[19]:
```
cd /mnt/d/Daneshga/Term\ 9/Omics\ TA/Sessions/
ls
```

```
1  2  3  4
```

[20]:
```
cd 3/
ls
```

```
Linux_Handbook.pdf  file1.txt  file2.txt
file3.txt  file4.txt
```

[21]:
```
cd ../
ls
```

```
1  2  3  4
```

[22]:
```
cd ~
ls
```

```
'MEGA X'           Untitled2.ipynb   env1.yml
miniconda3   perl5
 Untitled.ipynb   anaconda3         igv
mydir        temp-libraries
```

[23]:
```
cd /mnt/d/Daneshga/Term\ 9/Omics\ TA/Sessions/4/
```

1.4 4. Creating Files — touch

The touch command is used to **create empty files** or **update the timestamp** (access and modification time) of existing ones.

Basic syntax: “‘bash touch [options] filename

[25]:
```
touch script.sh
```

[26]:
```
touch text1.txt text2.txt
```

[27]:
```
ls
```

```
EssentialsOfLinux.ipynb  script.sh
text1.txt  text2.txt
```

1.5 5. Creating Directories — `mkdir`

The `mkdir` (**make directory**) command is used to **create new directories** (folders).

Basic syntax: “‘bash mkdir [options] directory_name

[28]:
```
mkdir dir1
ls
```

```
EssentialsOfLinux.ipynb  dir1  script.sh
text1.txt  text2.txt
```

[29]:
```
mkdir dir2 dir3
ls
```

```
EssentialsOfLinux.ipynb  dir1  dir2
dir3  script.sh  text1.txt
text2.txt
```

[30]:
```
mkdir /mnt/d/Daneshga/Term\ 9/Omics\ TA/Sessions/3/dir4/
ls /mnt/d/Daneshga/Term\ 9/Omics\ TA/Sessions/3/
```

```
Linux_Handbook.pdf  dir4  file1.txt
file2.txt  file3.txt  file4.txt
```

1.6 6. Copying Files and Directories — `cp`

The `cp` (**copy**) command is used to **copy files or directories** from one location to another.

Basic syntax: “‘bash cp [options] source destination

[32]:
```
cp text1.txt dir1
ls dir1/
```

```
text1.txt
```

[33]:
```
cp text2.txt /mnt/d/Daneshga/Term\ 9/Omics\ TA/Sessions/3/dir4/
ls /mnt/d/Daneshga/Term\ 9/Omics\ TA/Sessions/3/dir4/
```

```
text2.txt
```

[34]:
```
ls
```

```
EssentialsOfLinux.ipynb  dir1  dir2
dir3  script.sh  text1.txt
text2.txt
```

[35]:
```
cp -r dir1 dir2
ls dir2/
```

```
dir1
```

1.7 7. Moving and Renaming Files — `mv`

The `mv` (**move**) command is used to **move** or **rename** files and directories.

Basic syntax: “‘bash mv [options] source destination

[37]:
```
mv text2.txt dir3/
ls
```

```
EssentialsOfLinux.ipynb  dir1  dir2
dir3  script.sh  text1.txt
```

[38]:
```
ls dir3/
```

```
text2.txt
```

[39]:
```
mv script.sh renamed_sctipt.sh
ls
```

```
EssentialsOfLinux.ipynb  dir1  dir2
dir3  renamed_sctipt.sh  text1.txt
```

1.8 8. Removing Files and Directories — `rm`

The `rm` (**remove**) command is used to **delete files and directories**.

Warning: Once removed with `rm`, files are *not* sent to a trash or recycle bin — they are **permanently deleted**.
Always double-check what you’re deleting, and practice only on test files.

Basic syntax: “‘bash rm [options] file_name

[42]:
```
rm renamed_sctipt.sh
ls
```

```
EssentialsOfLinux.ipynb  dir1  dir2
dir3  text1.txt
```

[45]:
```
ls *
```

```
EssentialsOfLinux.ipynb  text1.txt

dir1:
text1.txt

dir2:
dir1

dir3:
text2.txt
```

[46]:
```
rm -rf dir3/
ls *
```

```
EssentialsOfLinux.ipynb  text1.txt

dir1:
text1.txt

dir2:
dir1
```

1.9 9. Removing Empty Directories — `rmdir`

The `rmdir` (**remove directory**) command is used to **delete empty directories**.
Unlike `rm -r`, it will *only* remove directories that contain no files or subdirectories — making it a safer option.

Basic syntax: “‘bash rmdir [options] directory_name

[47]:
```
rmdir dir2/
```

```
rmdir: failed to remove 'dir2/': Directory not empty
```

[48]:
```
mkdir empty_dir
ls
```

```
EssentialsOfLinux.ipynb  dir1  dir2
empty_dir  text1.txt
```

[49]:
```
rmdir empty_dir
ls
```

```
EssentialsOfLinux.ipynb  dir1  dir2
text1.txt
```

1.10 10. Checking Your Current Location — `pwd`

The `pwd` (**print working directory**) command displays the **full path** of the current directory you’re in.

Basic syntax: “‘bash pwd

[50]:
```
pwd
```

```
/mnt/d/Daneshga/Term 9/Omics TA/Sessions/4
```

[51]:
```
ls
```

```
EssentialsOfLinux.ipynb  dir1  dir2
text1.txt
```

1.11 11. Displaying Text and Variables — `echo`

The `echo` command is used to **print text or variables** to the terminal.
It's often used to: - Display short messages - Write text into files - Check environment variable values

Basic syntax: "'bash echo [options] [string]

[6]:
```
echo "Hello, Omics Students!"
```

```
Hello, Omics Students!
```

[7]:
```
echo -e "Line 1\nLine 2\n\tIndented Line 3"
```

```
Line 1
Line 2
        Indented Line 3
```

[8]:
```
touch text3.txt
ls
```

```
EssentialsOfLinux.ipynb  dir1  dir2
text1.txt  text3.txt
```

[9]:
```
echo -e "Line 1\nLine 2" > text3.txt
```

[11]:
```
echo "This is the first line." > text1.txt
# Append another line
echo "This is an appended line." >> text1.txt
```

1.12 12. Viewing File Contents — `cat`

The `cat` (**concatenate**) command is used to **display**, **combine**, or **create** text files directly in the terminal.

Basic syntax: "'bash cat [options] file_name

[12]:
```
cat text1.txt
```

```
This is the first line.
This is an appended line.
```

[13]:
```
cat text1.txt text3.txt
```

```
This is the first line.
This is an appended line.
```

```
Line 1
Line 2
```

[14]:
```
cat text1.txt text3.txt > text4.txt
```

[16]:
```
cat text4.txt
```

```
This is the first line.
This is an appended line.
Line 1
Line 2
```

[17]:
```
cat -n text4.txt
```

```
     1  This is the first line.
     2  This is an appended line.
     3  Line 1
     4  Line 2
```

1.13 13. Viewing the Beginning of Files — `head`

The `head` command displays the **first few lines** of a file (by default, the first 10 lines). It's useful for quickly inspecting the top portion of large files.

Basic syntax: "'bash head [options] file_name

[18]:
```
cd /mnt/d/NGS/Samples/P1/Reads/
ls
```

```
SRR062634_1.filt.fastq.gz     SRR062634_2.filt.fastq
hi.txt
SRR062634_1.filt_fastqc.html
SRR062634_2.filt_fastqc.html  sample.fastq
SRR062634_1.filt_fastqc.zip
SRR062634_2.filt_fastqc.zip
```

[19]:
```
head SRR062634_2.filt.fastq
```

```
@SRR062634.1 HWI-EAS110_103327062:6:1:1092:8469/2
CAGAAGAATGTAAACTCCAGGGGGACAGGAGTGCGTGCCGTGTGANTACCGGCTCNCCCTCTGTTCAATGGGGCGATCAC
CAGCACCTGTAACTTGCCTC
+
A>--?=C;C:=C=CC=D5D?C-:CAC:-
5=;>?'6>@########!#########!##############################################
@SRR062634.2 HWI-EAS110_103327062:6:1:1107:21105/2
CTGTCTCATAAAATAAAATAAAATAAAATAAATAAAATAAAATAAAATAAAATAAAATAAAATAAAATAAAATACCTTAT
TGCTAAAAAGGTGCTAACAA
+
DEEDAEEEEE:EB>E<>CCEEEBEEE<EDEAEE;EC<CEACAE;A>EEE@EE@E8>CEDA?C8DA68>D3B:@EE=-
```

```
AAC-CA5CDBDBE?:C@?BC?,?
@SRR062634.3 HWI-EAS110_103327062:6:1:1110:17198/2
TCAGGTGAATAAATTGAGAACAAATCTTGGAATCATGGAGAATGGATAGTCCAGAAAGAAAGATCACACGTTTTCTGCAA
GGAGAAACAGTATCCTCATT
```

[21]:
```
head -n 40 SRR062634_2.filt.fastq
```

```
@SRR062634.1 HWI-EAS110_103327062:6:1:1092:8469/2
CAGAAGAATGTAAACTCCAGGGGGACAGGAGTGCGTGCCGTGTGANTACCGGCTCNCCCTCTGTTCAATGGGGCGATCAC
CAGCACCTGTAACTTGCCTC
+
A>--?=C;C:=C=CC=D5D?C-:CAC:-
5=;>?'6>@########!#########!##############################################
@SRR062634.2 HWI-EAS110_103327062:6:1:1107:21105/2
CTGTCTCATAAAATAAAATAAAATAAAATAAATAAAATAAAATAAAATAAAATAAAATAAAATAAAATAAAATACCTTAT
TGCTAAAAAGGTGCTAACAA
+
DEEDAEEEEE:EB>E<>CCEEEBEEE<EDEAEE;EC<CEACAE;A>EEE@EE@E8>CEDA?C8DA68>D3B:@EE=-
AAC-CA5CDBDBE?:C@?BC?,?
@SRR062634.3 HWI-EAS110_103327062:6:1:1110:17198/2
TCAGGTGAATAAATTGAGAACAAATCTTGGAATCATGGAGAATGGATAGTCCAGAAAGAAAGATCACACGTTTTCTGCAA
GGAGAAACAGTATCCTCATT
+
:B5DDBDD:5DD6;:D:D5DBD6?ADD-
DD=B5DAB=5@A5AC5A--=::-C:5>5A2+C-)9(C;C>?>/)B)>?B?######################
@SRR062634.4 HWI-EAS110_103327062:6:1:1112:12923/2
ATCATTCCACAGGGTGGATCCTGCTTCTTAGAAAGATAAGGTATGCTATAAGGAGTCTCATTTCTCGATATTATAAGATT
ACGAGTACTAATACACCAAA
+
DD:??DD:5DA-5A5=6+?CC5=C-A<CA*.@<@@(5B=@D:B:D=-5AA:=:=,5A:B??::-
4(9;<;:>'B5BD:BB5:C:@5:->?B5C:?#####
@SRR062634.5 HWI-EAS110_103327062:6:1:1113:19453/2
GGCAAGAACTGGGAAGGAACAAATTCAAGTGGGCTTTTAGCATAGTAAGAGAACACTGTCCTTCAGTCTGATCATTTCAG
GGAGGGAGGGTATTTTTGTA
+
B?D:C=5DDDDDDBBCC:DCCC3=DB55DDBBDBCD:A?D=-@==-8.@74):266;=<<:BAB,??B@@->==-
??B5==>'@@################
@SRR062634.6 HWI-EAS110_103327062:6:1:1119:20104/2
ATCAGAGGTTTACAAACTCTAAGTCACAAAATCAGCTCACAGGCCACTATAAAATAGTCATTTATACAGACAAAAAGATC
AAACGAACGCATTTACACTA
+
:-D5=C=C=::AA>ABDD?=A-A?BC?55CDA?=?5BB?@-
@C5CB=?B?:BC=AB?4AA########################################
@SRR062634.7 HWI-EAS110_103327062:6:1:1123:15985/2
TTAAAAAAAAACATGGTGGACACAGGAAGGGGAACAACATACAGGGGCCTGTCAGGGGGGAGGGGGGGGGGAGAAAACCA
GGATAAATAGGTCAAGCCTG
+
AA?C-5C68=>B@BEDDAE6D:DDDF:5=FA:-
```

```
DDEE;5EE?5=?::=5A##################################################
@SRR062634.8 HWI-EAS110_103327062:6:1:1127:20916/2
TTGAGAGAATAAACTTGCCTTTAGCCTCCTTTAATTTTTACCTTTTATCTTTGGCTGAGGAGACAATAGCAAAAACAAAC
AAACAAAAAAAAAATGTTTT
+
?BADDDDD5BDDD>DD:-DD:DB=C.@4>0>-;>>DDDDDCB-A:;44=@:CC?-
;;>6@#######################################
@SRR062634.9 HWI-EAS110_103327062:6:1:1131:13093/2
TCCTTTCCCTCAATTATATTAGTCAGGATTTATTCTTGGACCTATTAATACTGAAAAGACATTCCTGAAACGGGTGAGAT
AGATTAATTTACAATTTAGC
+
EED-EC55ACC=:-DD:==?DD:D5?>@-?5?C;CBEED-
CC=C-@@;.B==D?::-AA-4;C@=E5E??D7:D:DC?A5->-<ACB?=BBB-BB<--AC
@SRR062634.10 HWI-EAS110_103327062:6:1:1134:19132/2
TACTTTTATCATATAGTTCCTGCCTTACCCACTCTAACACATATCCACATTTTGTTTATATGTAACATATAACAAGTAAA
GATATTCATCTCTGCCACAA
+
-A?A85?-/<<::D:-B5>:?A?D:>A->--::B5D5DD:=DD=-5CCC:.==?B*@<A6=@4,->6@-C-+-++>>-
A5C=C@C,:?C>>C-@######
```

1.14 14. Viewing the End of Files — `tail`

The `tail` command displays the **last few lines** of a file (by default, the last 10 lines).
It's often used to view the most recent entries in log files or outputs.

Basic syntax: "'bash tail [options] file_name

[22]:
```
tail SRR062634_2.filt.fastq
```

```
+
CCCC@=DBC5D?CCDDEEEDBC::-
AA=::@>@A5D??=B:@5=@+@?5??A:5?5>5>66B#######################################
@SRR062634.24475987 HWI-EAS110_103327062:6:120:19764:21021/2
TGATTCGGATTAGTTTCCCAGGCAGAACTGATTCCATCCCATCCCTGGCCTATCACATCCTCCCTACCCAAGAAATCTGT
AATGAAGTATGTTAAGTGCA
+
FFAFDFFFFFGGGGGDEBFGDGBGEFBCFDGGGEBDGEGGGCEEA?FFDACEDC,?BB@?DFDDFE=E5BEDCBEEDEGD
FBFF=:B?ABDEFE=C?C=E
@SRR062634.24475988 HWI-EAS110_103327062:6:120:19764:15802/2
AAAATAAAAGCTGCTTGGGCAAATTGTTAAACGTTTGCTTCTGTTCCTGAGATACCCAACTGGTCCAGAATTTTTTTTAG
GATACATTATTAGATTTACT
+
FGGGGGDGGFGGFGGGDFGDGDGGGGGFGDGGGGGGGDGGEGGGGGFGFGGFGGGGBG5GDEAEEFFFFFGGAGGGGG6E
E:C5CDEFFCFEFDFED:DE
```

[23]:
```
tail -n 40 SRR062634_2.filt.fastq
```

```
@SRR062634.24475979 HWI-EAS110_103327062:6:120:19763:4585/2
CAGGGCAGCTCTGGGCCAGCCCAAGGAGGCCTGGAATGCACACTTTACAGATGGGGAAACTGAGGCTGAAGAGCAGGGCG
```

```
CAGCGTTTTTCAAAGGGGAG
+
=FDFFCD5BD:?:AAA:CC@D=:5=D=DD-
AA5B?:A:>5:?:5A:4=>=-CC?CCA:=(4;.=:=65..A:,>?A:D?D####################
@SRR062634.24475980 HWI-EAS110_103327062:6:120:19763:15883/2
CGGGAACCTGTAATCCCAGCTATTCAAGAGGCTGAGGCAGGAGAATTGCTTAAAGCCGGGAGGTGGAGATTGCAGTGAGG
CAAGATCATGTCATTGTACT
+
GGBGEGFGGGGGGGGGGGFGGGFGGGGGGGAGGGGGGEGFGGG?FGGFGGGGGGEGGEFDBEB7EEEEC=CCBCE,CACC
=A?CACEB=?A?A5BCCBC?
@SRR062634.24475981 HWI-EAS110_103327062:6:120:19763:10207/2
AGATAATCGTGGAGAATTTTTGTGGGAAAGGATTGTAACACAAGAACTCTGTGTTCAGCTATGTCATTTTGTATGTGTGA
AGATGCCGGAAAGACATCAA
+
=B-DD?EEEEEE-ED=BDEBEE@CEEAFDFE5EEED:D?:BAEBEBBDCD?CCCCFEFDF@ADD5.CC@@EE4EDDDDB5
.@2;@->AAAA?CC8@B-A?
@SRR062634.24475982 HWI-EAS110_103327062:6:120:19763:4829/2
ATAGGCACCTGCCACCACGCCTGGCTAATTTTTGTATTTTTAGTAGAGATGGGGTTTCACCATGTTGGCCAGGATGAGCA
CAGACATTTTAAAATGGAAA
+
EGGGGEFFFDEEEDEFFBFEEFGGDGEGGGGGGDCDGGGGGGDGGGGGEGEEEECFFEFFDGGGGGGEDE?FFAFFD=E:
EEEBADFGFG?@B?ECD?=A
@SRR062634.24475983 HWI-EAS110_103327062:6:120:19764:15155/2
CAACTCATAGAACAGAACTATTAATTGAATTCAGCAAGGTCACAGGATACAAGATCAACATAGAAAAGTTAATTATTTTT
CTATGTATTAGTAATGAACA
+
GGGGGGGGGAGGGGGFGGGFGGGGGGGGGGGGDGGGDGGGEGGGGGDAGGDGGGG?FFFFGGGGGEDDED@DDDDFFEEE
GFGGDGGEDFGBGGGEF:FF
@SRR062634.24475984 HWI-EAS110_103327062:6:120:19764:7291/2
AAACTCCGTCTCCAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAATGAATAAAAAACCCCCCAAAAAAAAAA
AGAGACATCCCAAAATTCCA
+
FFEFFADFGGGGEDGFGEGDGFGGGGFAGA##########################################################
####################
@SRR062634.24475985 HWI-EAS110_103327062:6:120:19764:18294/2
ATGAGCCACCGTGGCCGGCCACTAATTTCTTATTATTGTTACAACTCTTCTTCCTTCTACTTGTTATTCCAGTTCTGCCA
TTTCCCAGCATTATAGCATT
+
GDEGGFGDEFEEEE?E:E:?:5??BECC5DA(CC6AC8>AB::EEB=E=AA?AAACCC,5A@@>46CA==5???,=>>>:
?@?:;CBB@BAEA5+<:<<:
@SRR062634.24475986 HWI-EAS110_103327062:6:120:19764:13120/2
TAAACAGTCCACTAACCCTGCCGATAACTATCATCTGGCCCGGAGAGATCGGAAGAGCGTCGTGTAGGGAAAGAGTGTAG
ATCTAGGTGGTTGGCGGGTC
+
CCCC@=DBC5D?CCDDEEEDBC::-
AA=::@>@A5D??=B:@5=@+@?5??A:5?5>5>66B#######################################
@SRR062634.24475987 HWI-EAS110_103327062:6:120:19764:21021/2
TGATTCGGATTAGTTTCCCAGGCAGAACTGATTCCATCCCATCCCTGGCCTATCACATCCTCCCTACCCAAGAAATCTGT
```

```
AATGAAGTATGTTAAGTGCA
+
FFAFDFFFFFGGGGGDEBFGDGBGEFBCFDGGGEBDGEGGGCEEA?FFDACEDC,?BB@?DFDDFE=E5BEDCBEEDEGD
FBFF=:B?ABDEFE=C?C=E
@SRR062634.24475988 HWI-EAS110_103327062:6:120:19764:15802/2
AAAATAAAAGCTGCTTGGGCAAATTGTTAAACGTTTGCTTCTGTTCCTGAGATACCCAACTGGTCCAGAATTTTTTTTAG
GATACATTATTAGATTTACT
+
FGGGGGDGGFGGFGGGDFGDGDGGGGGFGDGGGGGGGDGGEGGGGGFGFGGFGGGGBG5GDEAEEFFFFFGGAGGGGG6E
E:C5CDEFFCFEFDFED:DE
```

1.15 15. Counting Lines, Words, and Characters — `wc`

The `wc` (**word count**) command displays counts of **lines, words, and characters (or bytes)** in text files.

Basic syntax:

```
wc [options] file_name
```

Default output format: "' lines words bytes filename

[24]:
```
ls
```

```
SRR062634_1.filt.fastq.gz     SRR062634_2.filt.fastq
hi.txt
SRR062634_1.filt_fastqc.html
SRR062634_2.filt_fastqc.html  sample.fastq
SRR062634_1.filt_fastqc.zip
SRR062634_2.filt_fastqc.zip
```

[27]:
```
wc sample.fastq
```

```
 1000000  1250000 64905006 sample.fastq
```

[28]:
```
wc -l sample.fastq
```

```
1000000 sample.fastq
```

1.16 16. Creating Command Shortcuts — `alias`

The `alias` command lets you create **custom shortcuts** for other commands.
It's useful for saving time, avoiding long command sequences, or adding safety flags (like `-i` for interactive mode).

Tip: Aliases created this way are **temporary** — they last only for the current session. To make them permanent, add them to your ~/.bashrc or ~/.bash_aliases file.

Basic syntax: "'bash alias name='command'

[29]:
```
ls -al
```

```
total 8155744
drwxrwxrwx 1 mahdi mahdi       4096 Oct  6 09:55 .
drwxrwxrwx 1 mahdi mahdi       4096 Jan 10  2024 ..
-rwxrwxrwx 1 mahdi mahdi 1938959399 Dec 19  2023
SRR062634_1.filt.fastq.gz
-rwxrwxrwx 1 mahdi mahdi     632795 Jan  9  2024
SRR062634_1.filt_fastqc.html
-rwxrwxrwx 1 mahdi mahdi     446531 Jan  9  2024
SRR062634_1.filt_fastqc.zip
-rwxrwxrwx 1 mahdi mahdi 6345444769 Dec 19  2023
SRR062634_2.filt.fastq
-rwxrwxrwx 1 mahdi mahdi     633858 Jan  9  2024
SRR062634_2.filt_fastqc.html
-rwxrwxrwx 1 mahdi mahdi     448442 Jan  9  2024
SRR062634_2.filt_fastqc.zip
-rwxrwxrwx 1 mahdi mahdi          8 Oct  6 09:50 hi.txt
-rwxrwxrwx 1 mahdi mahdi   64905006 Oct  6 09:55 sample.fastq
```

[30]:
```
alias ll='ls -al'
```

[31]:
```
ll
```

```
total 8155744
drwxrwxrwx 1 mahdi mahdi       4096 Oct  6 09:55 .
drwxrwxrwx 1 mahdi mahdi       4096 Jan 10  2024 ..
-rwxrwxrwx 1 mahdi mahdi 1938959399 Dec 19  2023
SRR062634_1.filt.fastq.gz
-rwxrwxrwx 1 mahdi mahdi     632795 Jan  9  2024
SRR062634_1.filt_fastqc.html
-rwxrwxrwx 1 mahdi mahdi     446531 Jan  9  2024
SRR062634_1.filt_fastqc.zip
-rwxrwxrwx 1 mahdi mahdi 6345444769 Dec 19  2023
SRR062634_2.filt.fastq
-rwxrwxrwx 1 mahdi mahdi     633858 Jan  9  2024
SRR062634_2.filt_fastqc.html
-rwxrwxrwx 1 mahdi mahdi     448442 Jan  9  2024
SRR062634_2.filt_fastqc.zip
-rwxrwxrwx 1 mahdi mahdi          8 Oct  6 09:50 hi.txt
-rwxrwxrwx 1 mahdi mahdi   64905006 Oct  6 09:55 sample.fastq
```

1.17 17. Downloading Files from the Web — `wget`

The `wget` command is used to **download files** or web pages directly from the internet via the command line.

Basic syntax: “‘bash wget [options] [URL]

[33]:
```
cd /mnt/d/Daneshga/Term\ 9/Omics\ TA/Sessions/4
ls
```

```
EssentialsOfLinux.ipynb  dir1  dir2
text1.txt  text3.txt  text4.txt
```

[34]:
```
wget https://raw.githubusercontent.com/vincentarelbundock/Rdatasets/master/csv/
  ↪datasets/iris.csv
```

```
--2025-10-09 12:53:04--  https://raw.githubusercontent.com/vincentarelbundock/Rd
atasets/master/csv/datasets/iris.csv
Resolving raw.githubusercontent.com (raw.githubusercontent.com)…
185.199.111.133, 185.199.109.133, 185.199.108.133, …
Connecting to raw.githubusercontent.com
(raw.githubusercontent.com)|185.199.111.133|:443… connected.
HTTP request sent, awaiting response… 200 OK
Length: 4217 (4.1K) [text/plain]
Saving to: ‘iris.csv’

iris.csv            100%[===================>]   4.12K  --.-KB/s    in 0.005s

2025-10-09 12:53:05 (914 KB/s) - ‘iris.csv’ saved [4217/4217]
```

[35]:
```
ls
```

```
EssentialsOfLinux.ipynb  dir1  dir2
iris.csv  text1.txt  text3.txt
text4.txt
```

[36]:
```
head iris.csv
```

```
rownames,Sepal.Length,Sepal.Width,Petal.Length,Petal.Width,Species
1,5.1,3.5,1.4,0.2,setosa
2,4.9,3,1.4,0.2,setosa
3,4.7,3.2,1.3,0.2,setosa
4,4.6,3.1,1.5,0.2,setosa
5,5,3.6,1.4,0.2,setosa
6,5.4,3.9,1.7,0.4,setosa
7,4.6,3.4,1.4,0.3,setosa
8,5,3.4,1.5,0.2,setosa
9,4.4,2.9,1.4,0.2,setosa
```

1.18 18. Compressing and Decompressing Files — gzip

The gzip (**GNU zip**) command is used to **compress** or **decompress** files, reducing their size for storage or transfer.

Basic syntax: “‘bash gzip [options] file_name

[38]:
```
ls
```

```
EssentialsOfLinux.ipynb  dir1  dir2
iris.csv  text1.txt  text3.txt
text4.txt
```

[39]:
```
gzip text1.txt
ls
```

```
EssentialsOfLinux.ipynb  dir2
text1.txt.gz  text4.txt
dir1                     iris.csv  text3.txt
```

[40]:
```
gzip -k iris.csv
ls
```

```
EssentialsOfLinux.ipynb  dir2
iris.csv.gz   text3.txt
dir1                     iris.csv
text1.txt.gz  text4.txt
```

[50]:
```
gzip -r dir1/
```

[51]:
```
cd dir1/
ls
```

```
text1.txt.gz
```

[52]:
```
gzip -d text1.txt.gz
```

[53]:
```
ls
```

```
text1.txt
```

[54]:
```
cd ../
ls
```

```
EssentialsOfLinux.ipynb  dir2
iris.csv.gz   text3.txt
dir1                     iris.csv
text1.txt.gz  text4.txt
```

1.19 19. Displaying the Current User — `whoami`

The `whoami` command shows the **username** of the current user who is executing the command. It's useful when working on shared systems or remote servers (e.g., SSH sessions) to confirm your identity.

Basic syntax: "'bash whoami

[55]:
```
whoami
```

```
mahdi
```

1.20 20. Running Commands as Superuser — `sudo`

The `sudo` (**superuser do**) command allows a permitted user to **run commands with elevated (root) privileges**.
It's essential for performing administrative tasks like installing software, modifying system files, or managing users.

Be careful: Commands run with `sudo` have full system permissions. A small mistake can cause irreversible system changes.

Basic syntax: "'bash sudo [command]

[ ]:
```
sudo apt update
```

[ ]:
```
sudo apt upgrade
```

[ ]:
```
sudo apt install fastqc
```

1.21 21. Viewing Command History — `history`

The `history` command displays a **list of previously executed commands** in your shell session.
It's useful for recalling, repeating, or auditing commands.

Basic syntax: "'bash history [options]

[2]:
```
history 50
```

```
 1961  { echo $?; } 2>/dev/null
 1962  compgen -d -S / dir2
 1963  compgen -f dir2
 1964  compgen -abc -A function dir2
 1965  ls dir2/
 1966  { echo $?; } 2>/dev/null
 1967  ls dir1/
 1968  { echo $?; } 2>/dev/null
 1969  gzip -r dir1/
 1970  ls dir1/
 1971  { echo $?; } 2>/dev/null
 1972  compgen -d -S / dir1/
 1973  compgen -f dir1/
 1974  gzip -r dir1/
 1975  { echo $?; } 2>/dev/null
 1976  cd dir1/
 1977  ls
 1978  { echo $?; } 2>/dev/null
 1979  gzip -d text1.txt.gz
```

```
 1980  { echo $?; } 2>/dev/null
 1981  ls
 1982  { echo $?; } 2>/dev/null
 1983  cd ../
 1984  ls
 1985  { echo $?; } 2>/dev/null
 1986  whoami
 1987  { echo $?; } 2>/dev/null
 1988  sudo apt upgrade
 1989  jupyter notebook
 1990  cd /mnt/d/Daneshga/
 1991  jupyter notebook
 1992  PS1='PROMPT_RDXLYYSSQJCB\[\]>' PS2='PROMPT_RDXLYYSSQJCB\[\]+'
PROMPT_COMMAND=''
 1993  export PAGER=cat
 1994  bind 'set enable-bracketed-paste off' >/dev/null 2>&1 || true
 1995  display () {     display_id="$1"; shift;     TMPFILE=$(mktemp
${TMPDIR-/tmp}/bash_kernel.XXXXXXXXXX);     cat > $TMPFILE;
prefix="bash_kernel: saved image data to: ";     if [[ "${display_id}" != "" ]];
then         echo "${prefix}(${display_id}) $TMPFILE" >&2;     else         echo
"${prefix}$TMPFILE" >&2;     fi; }
 1996  displayHTML () {     display_id="$1"; shift;     TMPFILE=$(mktemp
${TMPDIR-/tmp}/bash_kernel.XXXXXXXXXX);     cat > $TMPFILE;
prefix="bash_kernel: saved html data to: ";     if [[ "${display_id}" != "" ]];
then         echo "${prefix}(${display_id}) $TMPFILE" >&2;     else         echo
"${prefix}$TMPFILE" >&2;     fi; }
 1997  displayJS () {     display_id="$1"; shift;     TMPFILE=$(mktemp
${TMPDIR-/tmp}/bash_kernel.XXXXXXXXXX);     cat > $TMPFILE;
prefix="bash_kernel: saved javascript data to: ";     if [[ "${display_id}" !=
"" ]]; then         echo "${prefix}(${display_id}) $TMPFILE" >&2;     else
echo "${prefix}$TMPFILE" >&2;     fi; }
 1998  export NOTEBOOK_BASH_KERNEL_CAPABILITIES="image,html,javascript"
 1999  history
 2000  { echo $?; } 2>/dev/null
 2001  PS1='PROMPT_NTEGEZIHSGZP\[\]>' PS2='PROMPT_NTEGEZIHSGZP\[\]+'
PROMPT_COMMAND=''
 2002  export PAGER=cat
 2003  bind 'set enable-bracketed-paste off' >/dev/null 2>&1 || true
 2004  display () {     display_id="$1"; shift;     TMPFILE=$(mktemp
${TMPDIR-/tmp}/bash_kernel.XXXXXXXXXX);     cat > $TMPFILE;
prefix="bash_kernel: saved image data to: ";     if [[ "${display_id}" != "" ]];
then         echo "${prefix}(${display_id}) $TMPFILE" >&2;     else         echo
"${prefix}$TMPFILE" >&2;     fi; }
 2005  displayHTML () {     display_id="$1"; shift;     TMPFILE=$(mktemp
${TMPDIR-/tmp}/bash_kernel.XXXXXXXXXX);     cat > $TMPFILE;
prefix="bash_kernel: saved html data to: ";     if [[ "${display_id}" != "" ]];
then         echo "${prefix}(${display_id}) $TMPFILE" >&2;     else         echo
"${prefix}$TMPFILE" >&2;     fi; }
```

```
 2006  displayJS () {     display_id="$1"; shift;     TMPFILE=$(mktemp
${TMPDIR-/tmp}/bash_kernel.XXXXXXXXXX);     cat > $TMPFILE;
prefix="bash_kernel: saved javascript data to: ";     if [[ "${display_id}" !=
"" ]]; then         echo "${prefix}(${display_id}) $TMPFILE" >&2;     else
echo "${prefix}$TMPFILE" >&2;     fi; }
 2007  export NOTEBOOK_BASH_KERNEL_CAPABILITIES="image,html,javascript"
 2008  history -50
 2009  { echo $?; } 2>/dev/null
 2010  history 50
```

[ ]: